

TP 3 : Métriques d'évaluation et Validation Croisée

Compléments de Mathématiques 2 - L3 MIAHS

Correction

2026-02-02

Table of contents

1	Introduction	1
2	Préparation des données	2
3	Un classifieur simple : La Régression Logistique	2
4	Ensembles d'apprentissage et de test	3
4.1	Méthode Holdout	3
5	Métriques d'évaluation	3
5.1	Implémentation manuelle vs Packages	3
5.1.1	Fonctions "Maison"	3
5.1.2	Utilisation de Caret	4
5.2	Analyse de la courbe ROC	4
6	Validation Croisée (k-fold)	5

1 Introduction

L'objectif de ce TP est de formaliser l'évaluation d'un classifieur supervisé. Nous allons implémenter les métriques classiques (Accuracy, Recall, Precision) et explorer deux méthodes de partitionnement des données : le **Holdout** et la **K-fold Cross-Validation**.

2 Préparation des données

Nous utilisons le jeu de données Titanic pré-traité. Pour cette analyse, nous nous concentrons sur deux variables prédictrices : **Fare** (tarif) et **Fam** (taille de la famille).

```
library(dplyr)
library(tidyverse)
library(caret)
library(Metrics)
library(pROC)

# Chargement
data <- read.csv("titanic_pre_processed.csv", stringsAsFactors = TRUE)
y <- data$Survived
X <- data %>% select(Fare, Fam)
```

3 Un classifieur simple : La Régression Logistique

Nous définissons un classifieur basé sur la régression logistique avec un seuil de décision modulable.

```
myClassifieur <- function(X_train, y_train, X_test, threshold = 0.5) {
  data_train <- cbind(y_train, X_train)
  mod <- glm(y_train ~ ., data = data_train, family = binomial(link = "logit"))

  # Probabilité -> Classe binaire
  y_pred <- as.numeric(predict(mod, newdata = X_test, type = "response") > threshold)
  return(y_pred)
}
```

i Influence du seuil

Par défaut, le seuil est de **0.5**. En abaissant ce seuil (ex: 0.3), on devient plus “libéral” dans la prédiction de la classe positive (survie), ce qui augmente mécaniquement le Rappel (Recall) mais diminue souvent la Précision.

4 Ensembles d'apprentissage et de test

4.1 Méthode Holdout

Pour évaluer la capacité de généralisation, nous séparons les données en un ensemble d'entraînement (80%) et un ensemble de test (20%).

```
holdOut <- function(X, y, split = 0.8) {  
  set.seed(1234) # Reproductibilité  
  n <- nrow(X)  
  inTrain <- sample(1:n, size = round(split * n), replace = FALSE)  
  
  list(  
    X_train = X[inTrain, ], y_train = y[inTrain],  
    X_test  = X[-inTrain, ], y_test  = y[-inTrain]  
  )  
}  
  
split_data <- holdOut(X, y)  
y_pred <- myClassifier(split_data$X_train, split_data$y_train, split_data$X_test)
```

5 Métriques d'évaluation

5.1 Implémentation manuelle vs Packages

5.1.1 Fonctions "Maison"

```
myAccuracy <- function(y_true, y_pred) mean(y_true == y_pred)  
  
myRecall <- function(y_true, y_pred) {  
  sum(y_true == 1 & y_pred == 1) / sum(y_true == 1)  
}  
  
myPrecision <- function(y_true, y_pred) {  
  TP <- sum(y_true == 1 & y_pred == 1)  
  FP <- sum(y_true == 0 & y_pred == 1)  
  if ((TP + FP) == 0) return(0)  
  TP / (TP + FP)  
}
```

5.1.2 Utilisation de Caret

```
# Matrice de confusion détaillée
cm <- confusionMatrix(factor(y_pred), factor(split_data$y_test), positive = "1")
print(cm$table)
```

	Reference	
Prediction	0	1
0	103	51
1	7	14

```
print(cm$byClass[c("Sensitivity", "Specificity", "Balanced Accuracy")])
```

Sensitivity	Specificity	Balanced Accuracy
0.2153846	0.9363636	0.5758741

5.2 Analyse de la courbe ROC

La courbe ROC permet d'évaluer le classifieur indépendamment du seuil choisi.

```
myClassifierProb <- function(X_train, y_train, X_test) {
  mod <- glm(y_train ~ ., data = cbind(y_train, X_train), family = binomial)
  predict(mod, newdata = X_test, type = "response")
}

probs <- myClassifierProb(split_data$X_train, split_data$y_train, split_data$X_test)
roc_obj <- roc(split_data$y_test, probs)

plot(roc_obj, col = "#2c3e50", lwd = 3, print.auc = TRUE)
```

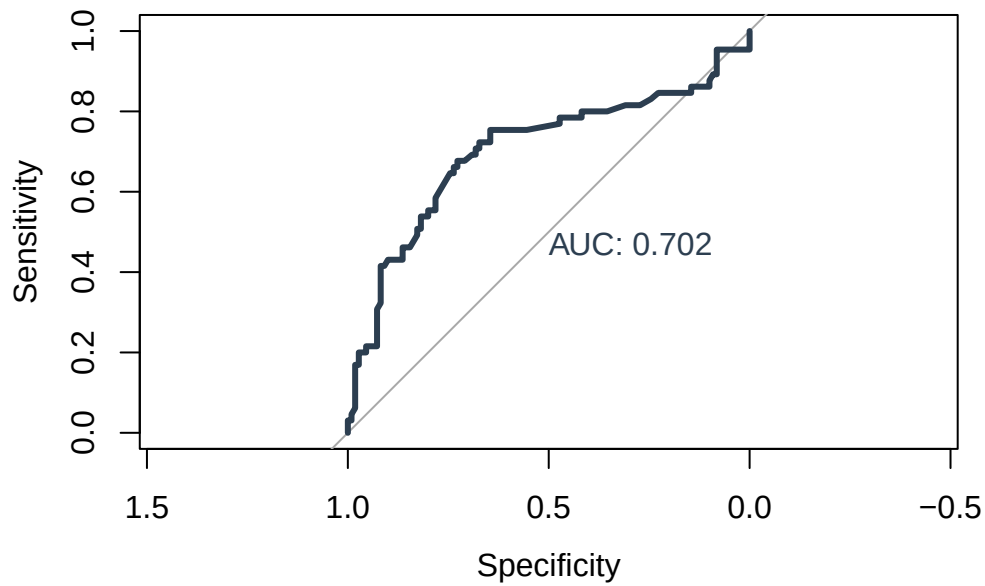


Figure 1: Courbe ROC du classifieur logistique

6 Validation Croisée (k-fold)

La validation croisée réduit le biais lié au découpage unique du Holdout en moyennant les scores sur segments.

```
run_cv <- function(X, y, k = 10) {
  set.seed(1234)
  folds <- sample(rep(1:k, length.out = nrow(X)))

  acc_results <- sapply(1:k, function(i) {
    X_tr <- X[folds != i, ]; y_tr <- y[folds != i]
    X_te <- X[folds == i, ]; y_te <- y[folds == i]

    preds <- myClassifier(X_tr, y_tr, X_te)
    myAccuracy(y_te, preds)
  })

  return(mean(acc_results))
}

avg_acc <- run_cv(X, y)
```

Résultat Final

L'accuracy moyenne obtenue par validation croisée est de **0.669**. Cette mesure est beaucoup plus robuste qu'un simple test unique.